

Tema 4 — Análisis Sintáctico

Procesamiento del Lenguaje Natural · UNIR · Guía de Estudio

1. Conceptos Fundamentales

Análisis Sintáctico (Parsing)

Proceso de determinar las *relaciones estructurales entre las palabras de una oración*. Su resultado es un **árbol sintáctico**, donde los nodos son constituyentes sintácticos y las hojas son las palabras individuales. Es un paso esencial entre el POS tagging y el análisis semántico.

Constituyente Sintáctico

Palabra o grupo de palabras que realiza una *función conjunta* dentro de la estructura jerárquica de la oración. Cuando tiene dos o más elementos se denomina **sintagma**.

Principales sintagmas del español

Sintagma	Símbolo	Núcleo	Funciones principales
Sintagma Nominal	SN	Sustantivo / Pronombre	Sujeto, CD, CI, atributo
Sintagma Verbal	SV	Verbo	Predicado verbal o nominal
Sintagma Preposicional	SPrep	Preposición	C. régimen, CD, CC
Sintagma Adjetival	SAdj	Adjetivo	C. adyacente, predicativo
Sintagma Adverbial	SAdv	Adverbio	Adjunto, CC

Reglas formales de estructura sintagmática

Las relaciones entre constituyentes se formalizan como reglas. Ejemplo:

$$SN \rightarrow Det N$$

Esta regla dice que un Sintagma Nominal puede estar formado por un determinante seguido de un nombre. Así, "la mesa" es un SN válido. Una oración completa típicamente se representa como:

$$S \rightarrow SN SV$$

2. Gramáticas de Estructura Sintagmática

Gramática Libre de Contexto (GLC)

Conjunto de reglas donde el lado izquierdo es siempre un único símbolo no terminal, y el lado derecho puede ser cualquier combinación de terminales y no terminales. "Libre de contexto" significa que la regla se aplica sin importar el entorno en que aparezca el símbolo.

Símbolos No Terminales vs. Terminales

- **No terminales:**

categorías abstractas que se pueden seguir expandiendo con reglas. Ejemplos:

$$S \quad , \quad SN \quad , \quad SV \quad , \quad Det \quad , \quad N \quad , \quad V \quad .$$

- **Terminales:**

las palabras concretas del vocabulario — ahí termina la derivación. Ejemplos: "mesa", "la", "cae".

Las gramáticas formales no fueron inventadas para PLN — las formalizó Chomsky en los años 50 como teoría matemática del lenguaje. El PLN las usa como *modelos computacionales* para que los algoritmos puedan procesar oraciones automáticamente.

3. Ambigüedad Estructural

Ambigüedad Estructural

Situación en que una oración admite *múltiples árboles sintácticos válidos* según las reglas de la gramática. No es un error de diseño — es inherente al lenguaje natural.

Ejemplo clásico — Groucho Marx

"I shot an elephant in my pajamas."

- **Lectura 1:**

"in my pajamas" modifica al SN "elephant" → el elefante estaba en los pijamas.

- **Lectura 2:**

"in my pajamas" modifica al SV "shot" → yo estaba en los pijamas al disparar.

Ambos árboles son gramaticalmente correctos. La gramática sola no puede resolver la ambigüedad.

Ejemplo en español

"hombres y mujeres mayores"

- [[hombres y mujeres] mayores] → todos son mayores
- [hombres y [mujeres mayores]] → solo las mujeres son mayores

La desambiguación puede hacerse **a posteriori** (después del análisis, usando conocimiento estadístico, semántico y contextual) o **integrada en el análisis** mediante métodos probabilistas.

4. Forma Normal de Chomsky (CNF)

Chomsky Normal Form — CNF

Restricción sobre las reglas de una GLC que exige que cada regla tenga exactamente una de estas dos formas:

$$A \rightarrow BC \quad (\text{dos símbolos no terminales})$$

$$A \rightarrow w \quad (\text{un símbolo terminal})$$

Cualquier GLC se puede transformar a CNF *sin perder expresividad*.

Transformaciones para convertir a CNF

Tipo de regla problemática	Transformación
Regla que mezcla terminales y no terminales $V \rightarrow book \ NP$	Introducir no terminal comodín: $V \rightarrow BOOK \ NP$ $BOOK \rightarrow book$ (genera 2 reglas, no 1)
Regla con más de dos símbolos a la derecha $S \rightarrow SN \ SV \ SPrep$	Binarizar con no terminal auxiliar: $S \rightarrow SN \ X1$ $X1 \rightarrow SV \ SPrep$

Regla unitaria (un solo no terminal a la derecha) $SN \rightarrow N$	Cortocircuitar: si $SN \Rightarrow N$ y $N \rightarrow mesa$, añadir directamente $SN \rightarrow mesa$ y eliminar la regla unitaria.
---	--

5. Algoritmo CKY — Cocke-Kasami-Younger

Programación Dinámica aplicada al Parsing

Técnica que resuelve subproblemas una sola vez y almacena los resultados, evitando recomputación. Principio fundamental: *"Las soluciones óptimas de subproblemas permiten encontrar la solución óptima del problema en su conjunto."* En parsing: los subproblemas son los constituyentes parciales de la oración.

Algoritmo CKY (Cocke-Kasami-Younger)

Algoritmo de análisis sintáctico basado en programación dinámica que requiere gramática en formato CNF. Trabaja con una **matriz triangular superior** de dimensión

$(n + 1) \times (n + 1)$ para una oración de n palabras.

Estructura de la matriz CKY

Cada celda $[i, j]$ contiene el conjunto de **símbolos no terminales** que representan todos los constituyentes sintácticos que abarcan posiciones desde i hasta j en la oración.

- **Superdiagonal** $[i - 1, i]$: categorías gramaticales de cada palabra individual (POS tags).
- **Diagonales superiores:** constituyentes para tramos de longitud *creciente*.
- **Celda** $[0, n]$: esquina superior derecha — abarca toda la oración. Si contiene S , la oración es gramaticalmente válida.

Pasos del algoritmo CKY

Paso 1 — Llenar la superdiagonal

Para cada palabra w_j , añadir a $[j - 1, j]$ todos los no terminales

A tales que $A \rightarrow w_j$ existe en la gramática. Una palabra ambigua (p.ej.

"book" = verbo o sustantivo) recibe *múltiples etiquetas simultáneamente*.

Paso 2 — Llenar las diagonales superiores (bottom-up, izquierda a derecha)

Para llenar $[i, j]$, el algoritmo prueba todos los puntos de corte k con

$i < k < j$:

- Si existe una regla $A \rightarrow BC$ en la gramática
- y $B \in [i, k]$ y $C \in [k, j]$
- entonces añadir A a $[i, j]$

El orden importa: $[i, k]$ y $[k, j]$ respetan la secuencia lineal de la oración.

Paso 3 — Recuperar el árbol

Para devolver un único árbol: elegir S de la celda $[0, n]$ y recuperar

recursivamente sus constituyentes siguiendo los punteros hacia las celdas de origen (backtrace — análogo al algoritmo de Viterbi del Tema 3).

Reconocedor vs. Analizador

El CKY básico es un *reconocedor* — determina si la oración es válida. Para convertirlo en *analizador* (que devuelve todos los árboles posibles) se hacen dos cambios:

1. Añadir punteros a las celdas de las que se derivó cada constituyente.
2. Permitir múltiples versiones del mismo símbolo no terminal en una celda.

Ejemplo numérico — "Book the flight through Houston"

Oración de 5 palabras $\rightarrow$ matriz 6×6 . La celda $[0, 1]$ contiene

$\{S, VP, Verb, Nominal, Noun\}$ porque "book" es ambigua (verbo

o sustantivo). La celda $[0, 5]$ contiene $\{S, VP, X_2\}$ — tres análisis

alternativos para la oración completa, reflejo de la ambigüedad estructural.

6. Métodos Probabilistas — CKY Probabilístico

Analizador Sintáctico Probabilista

Sistema que calcula la probabilidad de cada árbol sintáctico posible y selecciona el más probable. Formalmente, busca:

$$\hat{T}(S) = \underset{T \text{ t. q. } S = \text{yield}(T)}{\operatorname{argmax}} P(T)$$

donde $\text{yield}(T)$ es la cadena de palabras generada por el árbol T .

Probabilidad de un árbol sintáctico

Asumiendo **independencia entre reglas**, la probabilidad del árbol es el producto de las probabilidades de cada regla usada:

$$P(T) = \prod_i P(\text{regla}_i)$$

Ejemplo: si $P(S \rightarrow NP VP) = 0.80$,

$$P(NP \rightarrow Det N) = 0.30 \quad , \quad P(VP \rightarrow V NP) = 0.20 \quad ,$$

entonces $P(T) = 0.80 \times 0.30 \times 0.20 = 0.048$.

Cálculo de probabilidades de reglas

Método	Descripción	Requisito
Treebank	Contar expansiones y normalizar: $P(\alpha \rightarrow \beta \alpha) = \frac{Count(\alpha \rightarrow \beta)}{Count(\alpha)}$	Corpus anotado
Analizador no probabilístico	Analizar el corpus, contar reglas aplicadas, normalizar. Para oraciones ambiguas: ponderar por probabilidad del análisis.	Analizador no probabilístico + corpus
Inside-Outside (EM)	Partir de probabilidades iguales → analizar → recalcular → iterar hasta convergencia. Caso especial del algoritmo Expectation-Maximization.	Solo corpus (sin anotación)

CKY Probabilístico

Extensión del CKY básico donde cada regla está anotada con su probabilidad. La matriz pasa a ser de dimensión $(n + 1) \times (n + 1) \times V$, donde V es el número de símbolos no terminales. Cada celda almacena la *probabilidad* de cada constituyente para cada tramo de la oración.

En la conversión a CNF, las probabilidades de las reglas individuales se *recalculan* para que la probabilidad de cada árbol resultante permanezca constante.

7. Gramáticas de Dependencias

Gramática de Dependencias (Gramática Valencial)

Describe la estructura sintáctica de una oración únicamente en términos de las palabras y las *relaciones gramaticales* entre ellas. No hay nodos abstractos como SN o

SV — solo palabras conectadas por relaciones etiquetadas entre un **head** (núcleo) y un **dependent** (dependiente).

Comparación: Estructura Sintagmática vs. Dependencias

	Estructura Sintagmática	Dependencias
Modela	Constituyentes (grupos de palabras)	Relaciones entre palabras individuales
Resultado	Árbol con nodos abstractos (SN, SV...)	Grafo de palabras conectadas
Pregunta	¿Cómo se agrupan las palabras?	¿Qué palabra depende de cuál?
Idiomas con orden libre	Necesita reglas para cada orden posible	Agnóstica al orden

Información semántica	Se extrae con técnicas adicionales	Visible directamente en las relaciones
-----------------------	------------------------------------	--

Propiedades de la estructura de dependencias

Una gramática de dependencias forma un grafo $G = (V, A)$ donde V

son las palabras y A los arcos. Propiedades:

- Siempre hay un **único nodo raíz** sin arcos entrantes.
- Cada nodo (excepto la raíz) tiene un **único arco entrante** — un único head.
- Hay un **único camino** entre la raíz y cualquier otra palabra.

Proyectividad

Un arco head-dependent es *proyectivo* si existe un camino entre el head y todas las palabras que aparecen entre él y su dependent. Si todos los arcos son proyectivos, el árbol de dependencias es **proyectivo**.

Relaciones gramaticales principales

Etiqueta	Descripción
NSUBJ	Sujeto nominal
DOBJ	Objeto directo
IOBJ	Objeto indirecto
DET	Determinante
NMOD	Modificador nominal
AMOD	Modificador adjetival
CASE	Preposiciones y marcadores de caso

CONJ	Conjunción
------	------------

8. Transitioned-Based Dependency Parsing

Algoritmo de Transición para Dependencias

Construye el árbol de dependencias mediante tres acciones simples sobre una pila y un buffer:

- **SHIFT:**

toma la primera palabra del buffer y la pasa a la pila.

- **LEFTARC:**

establece relación head-dependent donde la primera palabra de la pila depende de la segunda. Elimina la primera de la pila.

- **RIGHTARC:**

establece relación head-dependent donde la segunda palabra de la pila depende de la primera. Elimina la primera de la pila.

El **oráculo** decide qué acción tomar en cada paso — se entrena con *aprendizaje supervisado* sobre treebanks.

9. Tabla Resumen Comparativa

Concepto	Característica clave
Análisis sintáctico	Determina estructura jerárquica → árbol sintáctico
Ambigüedad estructural	Inherente al lenguaje, múltiples árboles válidos
GLC	Conjunto de reglas para modelar la estructura
CNF	Restricción binaria: $A \rightarrow BC$ o $A \rightarrow w$
CKY básico	

	Programación dinámica, matriz $(n + 1)^2$, todos los árboles
CKY probabilístico	Añade $P(\text{regla})$, matriz $(n + 1)^2 \times V$, elige el más probable
Gramática de dependencias	Relaciones head-dependent entre palabras, sin nodos abstractos
Transitioned-based parsing	SHIFT / LEFTARC / RIGHTARC guiados por oráculo supervisado

10. Test de Autoevaluación — Claves

Claves del test oficial (10 preguntas)

Pregunta 1

— Ambigüedad en el análisis sintáctico

✓ C, D

A incorrecta: afecta a casi todos los analizadores. B incorrecta: la ambigüedad es inherente al lenguaje, no un error de diseño.

Pregunta 2

— Programación dinámica

✓ A, B, D

C incorrecta: no devuelve un único resultado — representa todos los árboles posibles.

Pregunta 3

— Algoritmo CKY

✓ A, B, D

C incorrecta: el CKY requiere CNF, no una GLC cualquiera. B correcta solo para la versión probabilística.

Pregunta 4

— Gramática CNF

✓ A, B, D

C incorrecta: la transformación de regla mixta genera *dos* reglas nuevas, no una.

Pregunta 5

— Funcionamiento del CKY

✓ A, B, C, D

Todas correctas.

Pregunta 6

— Matriz del CKY

✓ B, C

A incorrecta: la dimensión es $(n + 1) \times (n + 1)$, no $n \times n$. D

incorrecta: las diagonales superiores tienen longitud *creciente*, no decreciente.

Pregunta 7

— Reconocimiento sintáctico CKY

✓ B, C, D

A incorrecta: el reconocedor devuelve todos los árboles posibles, no uno único.

Pregunta 8

— Analizadores probabilistas

✓ A, B, C, D

Todas correctas.

Pregunta 9

— Cálculo de probabilidades de reglas

✓ B, C, D

A incorrecta: existen alternativas al treebank (analizador no probabilístico + EM).

Pregunta 10

— CKY probabilístico

✓ A, D

B incorrecta: las probabilidades de las reglas se *recalculan* (lo que se preserva es la probabilidad de cada árbol resultante). C incorrecta: la matriz es

$$(n + 1) \times (n + 1) \times V \quad , \text{no} \quad n \times n \quad .$$

Hilo conductor del Tema 4

Texto crudo → tokenización → POS tagging (Tema 3) → **Análisis Sintáctico** → Análisis Semántico. El problema central es la ambigüedad estructural, y el tema desarrolla tres soluciones: programación dinámica (CKY), probabilidades (CKY probabilístico) y un paradigma alternativo (gramáticas de dependencias).